Reinforcement Learning on the Taxi Environment: Tabular Q-learning and a DQN Comparison

Ana Abou-Arraj Awad Asier Ugarteche Perez

June 24, 2026

Abstract

We solve the Gymnasium `Taxi` environment with **tabular Q-learning** and study how hyperparameter choices affect performance. After a manually tuned baseline, we compare three automated tuning strategies — grid search, random search, and Hyperband (via Optuna). All four configurations ultimately solve the task with a 100% success rate, which illustrates a key point: on a small, deterministic environment like Taxi, hyperparameters influence *how fast* the agent learns more than *how well* it ends up performing. We close with a section on the **Deep Q-Network (DQN)** approach, which replaces the Q-table with a neural network — including a multi-seed hyperparameter study (network size, regularization, and the RL knobs) and a replay-buffer × target-network ablation — and discuss when that added machinery is warranted. As an optional extension we also apply the same DQN to Atari Pong from raw pixels, swapping the tabular-style MLP for a convolutional network and vanilla DQN for Double DQN.

1 The Taxi environment

Taxi is a 5×5 grid world. The agent drives a taxi to a passenger, picks them up, drives to a destination, and drops them off. The environment has:

- **500 states** — 25 taxi positions × 5 passenger locations (four pickup spots plus "inside the taxi") × 4 destinations;
- **6 actions** — South, North, East, West, Pickup, Dropoff;
- **rewards** — -1 per step, $+20$ for a correct dropoff, -10 for an illegal pickup/dropoff.

Crucially, Taxi is *deterministic*: a trained agent should deliver the passenger every single time. With only 500 discrete states, the problem is small enough that a lookup table over (state, action) pairs is a natural and sufficient representation — which is exactly what tabular Q-learning uses.

2 Tabular Q-learning (without DQN)

2.1 The algorithm

Q-learning stores one value $Q(s, a)$ per state–action pair in a 500×6 table, initialised to zero. At each step the agent takes an ϵ-greedy action and updates the table using the Bellman rule:

$$Q(s,a) \;\leftarrow\; Q(s,a) \;+\; \alpha \Big[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \Big]. \tag{1}$$

Exploration is annealed over training with an exponential schedule,

$$\epsilon(\text{ep}) \;=\; \epsilon_{\min} + (\epsilon_{\max} - \epsilon_{\min})\, e^{-\texttt{decay\_rate}\cdot \text{ep}}, \tag{2}$$

so the agent explores heavily early and increasingly exploits its learned policy later.

2.2 Hyperparameters

The behaviour of tabular Q-learning is governed entirely by a handful of hyperparameters. Table 1 lists each one, its symbol in Eqs. (1)–(2), its role, and the value used in the manual baseline (Part 1).

Hyperparameter	Symbol	Manual value	Role
`learning_rate`	α	0.7	step size of the Bellman update
`gamma`	γ	0.95	discount on future reward
`total_episodes`	—	25,000	number of training episodes
`max_steps`	—	200	step cap per episode
`max_epsilon`	$\epsilon_{\max}$	1.0	initial exploration rate
`min_epsilon`	$\epsilon_{\min}$	0.01	exploration floor
`decay_rate`	—	0.0005	speed of ϵ decay

Table 1: Q-learning hyperparameters and their manual-baseline values.

How each one matters.

- `learning_rate` (α): how strongly each new experience overwrites the current estimate. High values learn fast but can be unstable; low values are smoother but slower.
- `gamma` (γ): how much the agent values future reward. Close to 1 makes it far-sighted, which suits Taxi because the big +20 reward only arrives at the end of a successful trip.
- `min_epsilon` and `decay_rate`: together they shape the exploration-to-exploitation transition. A *low* `min_epsilon` lets the agent commit to its learned policy once it knows enough; `decay_rate` sets how quickly it gets there. These two turned out to be the most influential hyperparameters (Section 2.7).
- `total_episodes` and `max_steps`: the training budget and the per-episode horizon. 200 steps is comfortably above the ∼13 steps an optimal policy needs.

Evaluation note. Throughout, the trained Q-table is evaluated with a **pure greedy policy** ($\epsilon = 0$), averaged over 5 seeds × 200 episodes = 1,000 evaluation episodes per method. Because Taxi is deterministic, the only source of variance is the randomised initial state; averaging over seeds and episodes gives a stable, reproducible score.

2.3 Part 1 — Manual training

With the values in Table 1, the agent trained for 25,000 episodes. The learning curve (Figure 1) climbs steeply in the first few thousand episodes and then flattens once a solid policy is found. The evaluated baseline:

$$\text{avg reward } 7.93, \qquad \text{avg steps } 13.1, \qquad \text{success } 100\%.$$

2.4 Part 2 — Grid search

Grid search trains every combination of a fixed set of values: `learning_rate` $\in \{0.1, 0.3, 0.5, 0.7, 0.9\}$, `gamma` $\in \{0.85, 0.95, 0.99\}$, `decay_rate` $\in \{0.0005, 0.002\}$, and `min_epsilon` $\in \{0.01, 0.05\}$ — 60 combinations in total. Each was trained for 15,000 episodes and scored by the mean reward over the last 1,000 episodes. It is exhaustive but constrained to the predefined grid points. The best configuration scored 7.62:

$$\texttt{learning\_rate} = 0.9, \quad \texttt{gamma} = 0.99, \quad \texttt{decay\_rate} = 0.0005, \quad \texttt{min\_epsilon} = 0.01.$$

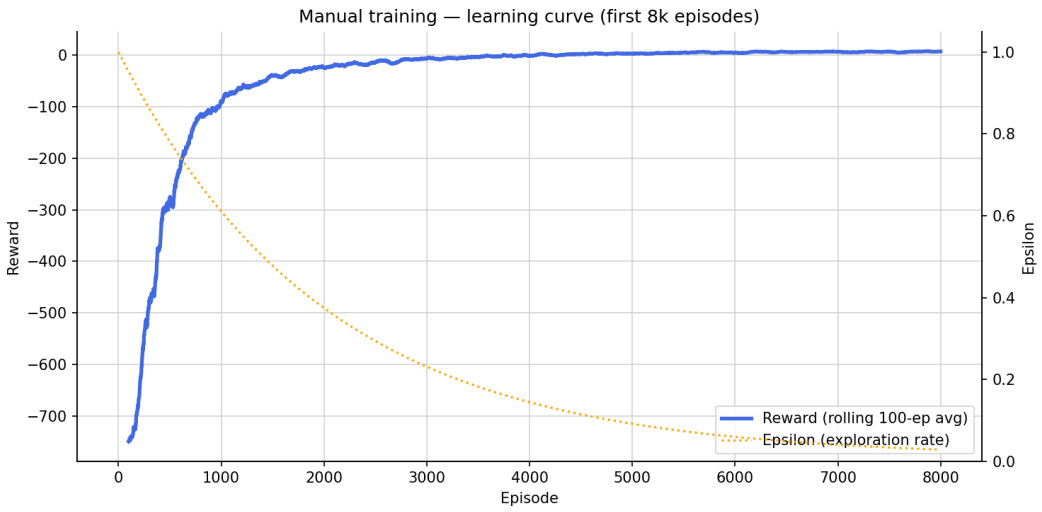

Figure 1: Manual-baseline learning curve (first 8,000 episodes, where learning happens): training reward as a 100-episode rolling average (left axis, blue) and the exploration rate ϵ (right axis, orange dotted). Reward climbs steeply over the first few thousand episodes and then flattens as ϵ decays and the agent shifts from exploring to exploiting. Code: `QLearning_Taxi.ipynb`, Part 1.

The dominant pattern was `min_epsilon`: every `min_epsilon`= 0.01 panel outperformed its `min_epsilon`= 0.05 counterpart by roughly 2–2.5 reward points, regardless of the other hyperparameters. Once `min_epsilon` is low, the environment is tolerant of `learning_rate`, `gamma`, and `decay_rate` — no single one of them produces a sharp peak.

2.5 Part 3 — Random search

Instead of a fixed grid, random search samples 30 configurations from continuous ranges: `learning_rate` $\sim U(0.1, 0.9)$, `gamma` $\sim U(0.80, 0.999)$, `decay_rate` $\sim U(0.0001, 0.005)$, `min_epsilon` $\sim U(0.001, 0.1)$. With the same number of trials it covers the space better than grid because it is not pinned to predetermined points. The best trial scored 7.92:

`learning_rate` $= 0.646$, `gamma` $= 0.828$, `decay_rate` $= 0.00108$, `min_epsilon` $= 0.00173$.

The scatter-matrix analysis again singled out `min_epsilon` as by far the strongest predictor: every top-10 trial had `min_epsilon` below 0.03, while every bottom-10 trial had it above 0.07 — the two groups barely overlap. `decay_rate` shows a much weaker secondary trend; `learning_rate` and `gamma` show no clean individual pattern.

2.6 Part 4 — Hyperband with Optuna

Grid and random search waste compute by training every configuration for the full budget, regardless of how it is doing. **Hyperband** runs the search like a tournament: it starts many configurations on a small budget, eliminates the worst, and gives more episodes only to the survivors. We implement this with **Optuna** (a TPE sampler combined with its `HyperbandPruner`) over the same continuous ranges as random search, for 40 trials — of which 31 were pruned early by Hyperband, with only 9 running to completion. The best trial scored 7.74:

`learning_rate` $= 0.888$, `gamma` $= 0.871$, `decay_rate` $= 0.0049$, `min_epsilon` $= 0.0021$.

Optuna's own hyperparameter-importance estimate (fANOVA over the completed trials) confirms the ordering seen in the grid and random searches (Figure 2): `min_epsilon` dominates (≈ 0.52), with the ϵ `decay_rate` a clear second (≈ 0.31) and `learning_rate` and `gamma` contributing little (≈ 0.09 each). In other words, how exploration is shut down — both its floor and how fast it gets there — matters far more than the learning rate or discount on this deterministic environment.

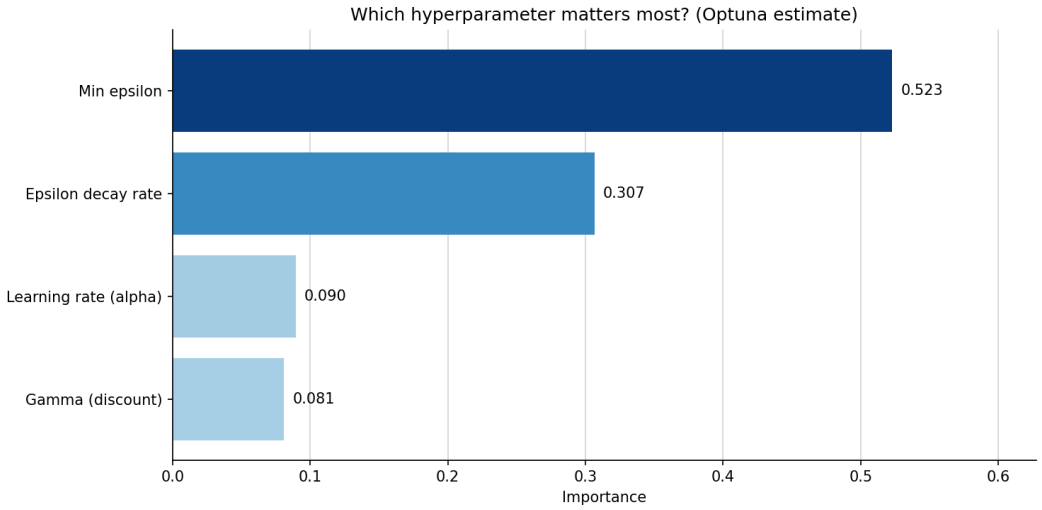

Figure 2: Optuna hyperparameter importance (fANOVA) over the 40-trial Hyperband study. `min_epsilon` is by far the strongest predictor of final score, followed by the ϵ `decay_rate`; `learning_rate` and `gamma` are near-negligible. Code: `QLearning_Taxi.ipynb`, Part 4.

2.7 Part 5 — Final comparison

The best configuration from each method was retrained for the full 25,000 episodes and evaluated identically. Table 2 summarises the best configurations and their search scores; Table 3 reports the final evaluation.

Plotting the four retrained runs side by side (Figure 3) shows that the methods differ in *how fast* they learn far more than in where they end up: all four climb to the same plateau, but the Optuna/Hyperband config converges fastest — it reaches near-optimal reward within roughly the first thousand episodes, whereas the manual and grid configs take several thousand. The reason is visible in Table 2: Optuna's config pairs a high learning rate with a much faster ϵ decay (0.0049 vs. 0.0005), so it stops exploring and commits to its policy early. Manual and grid share the slow decay 0.0005 and their curves almost coincide. This is the same lesson as before — on a deterministic environment with a unique optimal policy, tuning buys faster convergence rather than a better final policy.

Analysis. Three observations stand out:

1. **All four methods solve Taxi** (100% success, ∼13.1–13.2 steps), and their final rewards fall within 0.10 of each other. On a deterministic environment, hyperparameters affect *how fast* the agent converges far more than the final policy quality, since the optimal policy is unique and easily representable in a table.

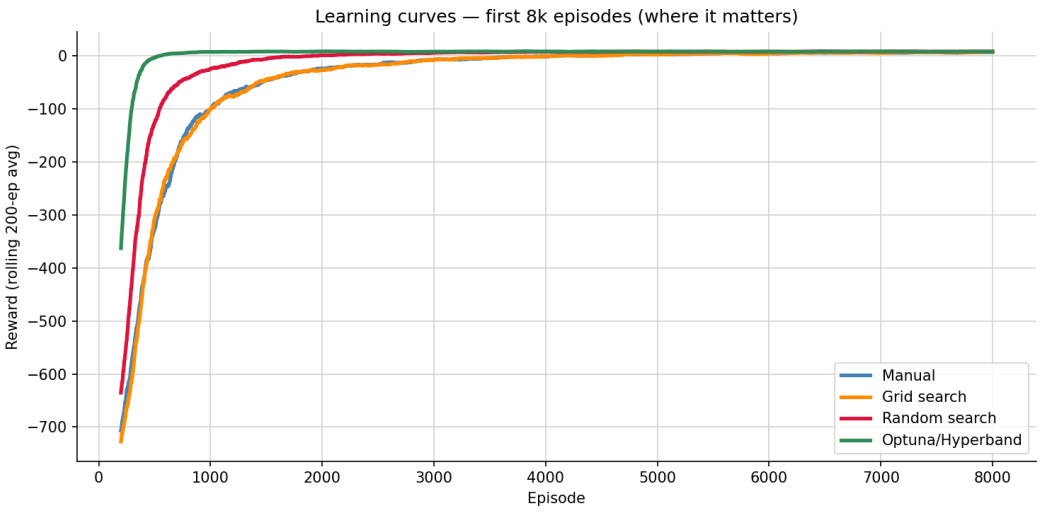

Figure 3: Learning curves (reward, 200-episode rolling average) for the best configuration of each method, retrained for 25,000 episodes and zoomed to the first 8,000 where the differences appear. All four reach the same plateau; Optuna/Hyperband (green) gets there fastest thanks to its faster ϵ decay, while manual (blue) and grid (orange) share the slow decay and nearly overlap. Code: `QLearning_Taxi.ipynb`, Part 5.

Method	α	γ	`decay_rate`	`min_epsilon`	**Search score**
Manual	0.70	0.95	0.0005	0.0100	—
Grid search	0.90	0.99	0.0005	0.0100	7.62
Random search	0.646	0.828	0.00108	0.00173	7.92
Optuna/Hyperband	0.888	0.871	0.0049	0.0021	7.74

Table 2: Best configuration found by each method (search score = mean reward over the last 1,000 of 15,000 training episodes for grid/random/Optuna; manual uses 25,000 episodes).

2. **During the search phase**, random search found the highest-scoring configuration (7.92), followed by grid (7.62) and Optuna (7.74). After full retraining for 25,000 episodes, those differences washed out: all four methods are statistically indistinguishable (7.83–7.93).
3. **Compute cost** is where the methods differ most meaningfully. Grid search consumed 900,000 episodes (60 combinations × 15,000). Random search used half that — 450,000 episodes — for a result within 0.01 reward of grid search's best. Optuna/Hyperband used only ∼164,000 episodes, pruning 31 of 40 trials early, and still landed within 0.10 of the top performer: roughly a **quarter** of grid search's compute for a comparable result.

The consistent thread across all searches is that `min_epsilon` is by far the most influential hyperparameter: a low exploration floor lets the agent commit fully to its learned policy. `decay_rate` has a weaker secondary effect; `learning_rate` and `gamma` show no clear independent pattern once `min_epsilon` is controlled for.

Method	Avg reward	Avg steps	Success rate
Manual	**7.93**	**13.1**	100%
Grid search	**7.93**	**13.1**	100%
Random search	7.92	13.1	100%
Optuna/Hyperband	7.83	13.2	100%

Table 3: Final evaluation after retraining each best config for 25,000 episodes (pure greedy, $\epsilon = 0$, averaged over 5 seeds $\times$ 200 episodes).

3 Deep Q-Network (DQN) approach

The natural question is what changes if we replace the Q-table with a neural network — a **Deep Q-Network**. The algorithm is identical (the Bellman update of Eq. (1)); only the *representation* of Q changes.

3.1 From a Q-table to a function

A Q-table stores one number per (s, a) pair — a lookup. A DQN instead *computes* $Q(s, a)$ with a neural network. Because the network cannot consume a raw state ID (the integer 499 is not "larger" than 12 in any meaningful sense), each Taxi state is converted to a **one-hot vector** of length 500 before being fed to the network. This makes the DQN on Taxi close to a tabular method in disguise: with one-hot inputs the network learns essentially one independent column of weights per state, so there is little generalisation across states.

Two ways to apply the same Bellman target. The methods also *update* differently. Tabular Q-learning edits one cell in place using the temporal-difference (TD) error — the bracketed quantity in Eq. (1), $\delta = r + \gamma \max_{a'} Q(s', a') - Q(s, a)$, scaled by the learning rate α. A neural network cannot edit individual outputs, so a DQN instead forms a fixed **bootstrap target** from the (frozen) target network,

$$y \;=\; r + \gamma \max_{a'} Q_{\theta^-}(s', a'),$$

and **minimises a regression loss** between its prediction and that target by gradient descent,

$$\mathcal{L}(\theta) \;=\; \big(Q_\theta(s, a) - y\big)^2 \qquad \text{(Huber / smooth-}L_1\text{ in practice)},$$

where θ is the online (policy) network and θ^- the target network. So the explicit subtraction of Eq. (1) becomes a squared (or Huber) loss the optimiser descends — the same Bellman principle, different machinery (in-place table edit vs. backprop on a function approximator).

3.2 Architecture and key components

The DQN solution[1] uses:

- a small multilayer perceptron, $500 \to 64 \to 64 \to 6$ (one-hot input, one Q-value per action);
- an **experience replay buffer** (capacity 50,000) that stores past transitions and samples random minibatches of 64, breaking the correlation between consecutive steps. Sampling reads transitions without removing them, so the same experience may be reused in later updates; only when the buffer reaches capacity does adding a new transition discard the oldest one;

[1] Full code and outputs: `DQN_Taxi.ipynb`.

- a **target network** — a slowly-updated copy of the policy network, synced every 250 steps — used to compute a stable Bellman target;
- the Huber (smooth-L_1) loss with gradient clipping, optimised by Adam.

These two stabilisers (replay buffer and target network) are the machinery that distinguishes DQN from plain online Q-learning with a function approximator.

3.3 Replay warm-up and per-step learning

The replay threshold is measured in *transitions*, not episodes. Although training runs for 500 episodes, the replay warm-up ends as soon as 500 individual environment steps have been stored, normally within the first few episodes because each episode can last up to 200 steps. Before that threshold, the agent continues to act and populate the buffer, but no backpropagation or Q-value update occurs. Action selection is still called "ϵ-greedy": with probability ϵ the action is random, while with probability $1 - \epsilon$ it is the argmax of the policy network. During warm-up that network is still randomly initialised, however, so its rare "exploit" actions are not yet informed by learning.

After each environment step the new transition (s, a, r, s', d) is added first. Once the buffer contains at least 500 transitions, `train_frequency = 1` means that every subsequent step is followed by one optimizer update from a randomly sampled batch of 64 stored transitions. These samples do not empty the replay buffer. The target network is updated separately by copying the policy-network weights every 250 environment steps.

3.4 Baseline configuration and result

The baseline full DQN uses $\gamma = 0.99$, learning rate 10^{-3}, batch size 64, replay capacity 50,000, replay warm-up 500 transitions, one optimizer update per environment step after warm-up, and a target-update period of 250 steps. Its exponential exploration schedule approaches the floor 0.05 rather than reaching it within the run: ϵ is approximately 0.999 at episode 1, 0.868 at episode 100, and 0.499 at episode 500. This deliberately slow decay keeps collecting varied experience; it does not mean that the final policy is evaluated with 50% random actions. Training lasts 500 episodes, whereas evaluation is fully greedy ($\epsilon = 0$). The resulting agent reaches:

avg reward 8.04, avg steps 13.0, success 100%.

3.5 Hyperparameter study

Mirroring the tabular search, we tune the DQN in two stages (coordinate descent), each trained over **multiple seeds** ($\{7, 8, 9\}$) and reported as **mean** $\pm$ **std**. The multi-seed protocol is deliberate: with a single seed the *ranking* of near-equal variants can flip from run to run, so averaging over seeds (and reporting the spread) is what makes a "best" claim defensible.

Stage 1 — network size and regularization. Holding the full DQN fixed, we vary the architecture — a single 32-unit hidden layer, the 64-64 baseline, and a wider 128-128 — and the regularization (L2 weight decay 10^{-4}, dropout 0.1), each trained for 400 episodes over three seeds (Table 4; learning curves in Figure 4).

The standout is 128-128 **with weight decay**: it is the *only* variant to fully solve Taxi — **100**% success in the optimal $\sim$13 steps, with essentially zero variance ($\pm$0.04 over seeds). Weight decay's effect is **architecture-dependent**: it *rescues* the high-capacity wide net (from 94.7% to a perfect score) yet *hurts* the small 64-64 net (85.0% $\rightarrow$ 81.7%) — consistent with the larger network being

Variant	Params	Success % (mean)	Reward (mean ± std)
128-128 + weight decay	81,414	**100.0**	$\mathbf{7.99 \pm 0.04}$
wide 128-128	81,414	94.7	-3.11 ± 15.71
baseline 64-64	36,614	85.0	-23.15 ± 23.36
64-64 + weight decay	36,614	81.7	-29.93 ± 30.49
shallow 32	16,230	63.7	-67.31 ± 28.05
64-64 + dropout 0.1	36,614	0.0	-200.0 ± 0.0
128-128 + dropout 0.1	81,414	0.0	-200.0 ± 0.0

Table 4: Network / regularization variants — 3 seeds, 400 training episodes, greedy eval.

the one prone to the over-fitting/instability that regularization tames. Network size alone helps (wide 128-128 beats the 64-64 baseline beats the shallow 32), but only *with* weight decay does it become rock-solid. **Dropout is catastrophic** on both sizes (0%) — injecting noise into the Q-estimates breaks DQN. Finally, the winning 128-128+weight-decay matches the single-seed Part-1 figure (Section 3.4: 8.04 reward, 100%) but now *across three seeds with a* ± 0.04 *spread*: unlike the noisier variants (whose multi-seed averages sit well below that figure — a reminder that one lucky seed can flatter), this configuration is genuinely robust.

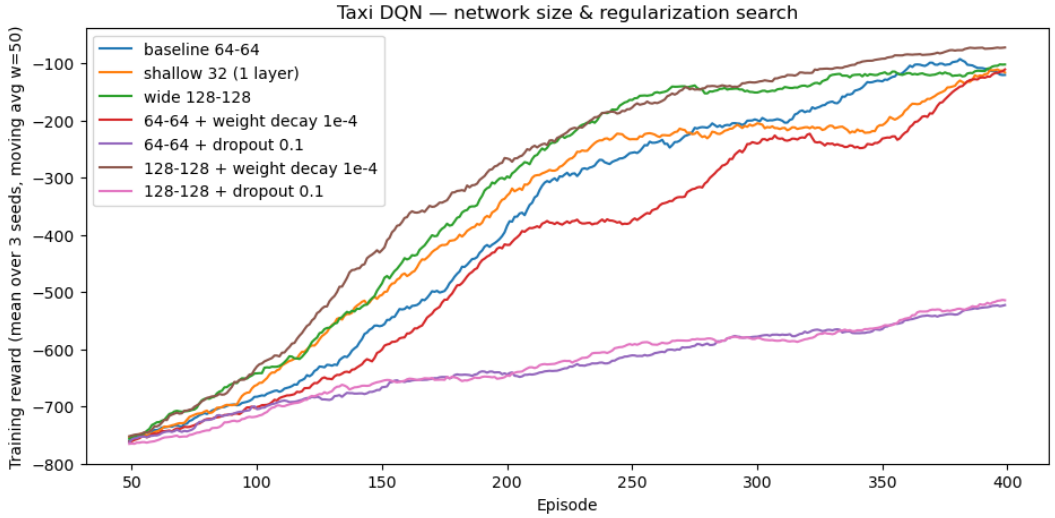

Figure 4: Stage 1 learning curves — training reward (mean over 3 seeds, 50-episode moving average) for the seven network/regularization variants. 128-128 + weight decay converges highest (to the optimal return); the two dropout variants flatline at the failure floor. Code: `DQN_Taxi.ipynb`, Part 2.1.

Stage 2 — RL hyperparameters. Table 5 lists the knobs we search — what each controls, the range it was sampled from, and its default (Part 1) value.

A random search of 16 configurations — over the learning knobs (learning rate, γ, ϵ-decay), the replay/optimisation settings (batch size, learning frequency), and the **target-update mechanism** (a hard periodic copy vs. a soft Polyak update $\theta^- \leftarrow \tau\,\theta + (1-\tau)\,\theta^-$) — found that *none* beats the defaults: no random trial outscored the default recipe in the single-seed scan, and the hard-vs-soft choice in particular made no difference, the default hard update already being best. Combined with

Knob	What it does	Search range	Default
`lr`	Adam step size — how fast the weights move	3×10^{-4}–5×10^{-3}	10^{-3}
`gamma` (γ)	discount on future reward (far-sightedness)	0.95–0.999	0.99
`decay`	rate at which exploration ϵ decays	5×10^{-4}–4×10^{-3}	1.5×10^{-3}
`batch_size`	transitions sampled per gradient step	{32, 64, 128}	64
`train_freq`	one gradient step every N environment steps	{1, 2, 4}	1
`memory`	replay-buffer capacity (transitions kept)	2,000–50,000 (log)	50,000
`update`	target-network update mechanism	hard / soft	hard
`target_update`	*(hard)* env steps between target syncs	100–1000	250
`tau` (τ)	*(soft)* fraction the target shifts toward the policy each step	0.001–0.02 (log)	—

Table 5: Stage 2 search space — the RL hyperparameters, what each controls, the range each was sampled from, and its default (Part 1) value.

Stage 1, the recommended configuration is therefore 128-128 **with weight decay** and the *default* RL hyperparameters, scoring $\mathbf{7.99 \pm 0.04}$ reward at **100**% success (the standout row of Table 4) — Taxi solved in the optimal ∼13 steps. The RL knobs are thus effectively *neutral*: the standard recipe is already best, and what lifts the agent to a perfect score is the *weight decay* from Stage 1, not the learning knobs.

The search *failures* are themselves informative. The lowest-scoring configurations share a 1:4 learning frequency (a gradient step only every fourth environment step — hence undertrained at the 250-episode scan) and/or a tiny replay buffer (∼2–3k transitions, prone to forgetting): what breaks the agent is *undertraining and forgetting*, not the learning-rate/γ/ϵ-decay knobs, which show no clean trend. The hard-vs-soft target choice in particular shows no pattern — both appear at the top *and* the bottom of the ranking. The defaults already avoid both failure modes (train every step, 50k buffer), which is why they are robust and nothing beats them. (Tendencies only: the scan is single-seed and varies every knob at once.) The full 16-trial search table is in the notebook (Part 2.2).

3.6 Ablation: replay buffer and target network

To isolate what each algorithm component contributes, we train all four corners of the *replay buffer* × *target network* 2×2 — both, buffer-only, target-only, and neither (online, batch size 1) — again over multiple seeds, reported as mean ± std. Importantly, the network and hyperparameters are held at a **neutral baseline** rather than the tuned best from Section 3.5: tuned settings become *co-adapted* to the full system, so reusing them would unfairly handicap the ablated corners (e.g. a learning rate that is stable only *because* of the target network would blow up the target-only corner). A shared neutral reference keeps the comparison fair.

Results. The outcome is unambiguous (Table 6, Figure 5). The **replay buffer is decisive**: without it (*target-only* and *neither*) the agent never learns the sparse reward and fails completely (0% success, −200 reward). With it (*both* and *buffer-only*) it solves Taxi at 87–91%. The **target**

network adds only a modest gain — *both* (91.3%) over *buffer-only* (87.0%) — so it is a useful refinement, not a necessity. On near-tabular Taxi, the replay buffer carries most of the load.

Variant	Replay	Target	Success % (mean ± std)	Reward (mean ± std)
both	yes	yes	91.3 ± 12.3	-10.06 ± 25.54
buffer only	yes	no	87.0 ± 16.3	-18.91 ± 33.77
target only	no	yes	0.0 ± 0.0	-200.0 ± 0.0
neither	no	no	0.0 ± 0.0	-200.0 ± 0.0

Table 6: Replay-buffer × target-network ablation — 3 seeds, 500 episodes, greedy eval.

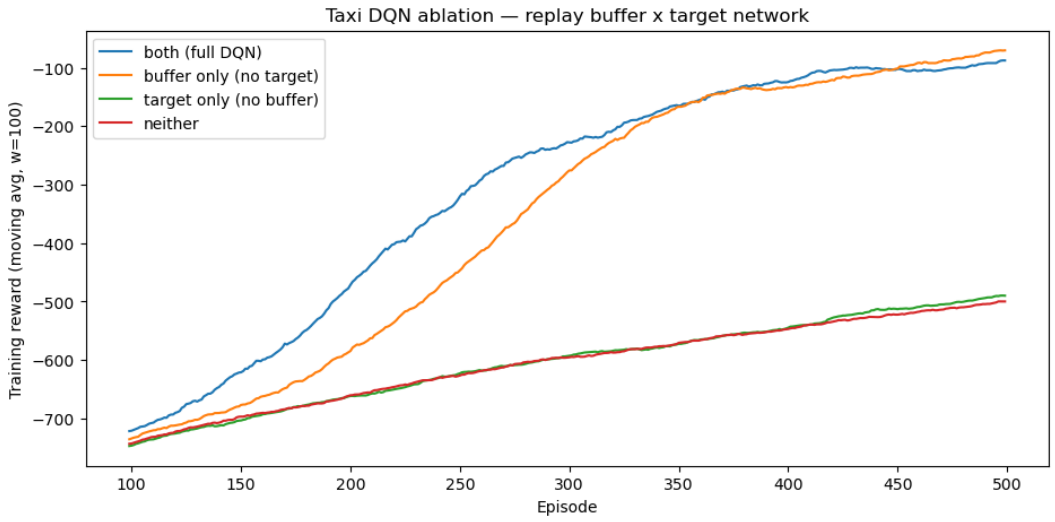

Figure 5: Ablation learning curves — training reward (mean over 3 seeds, 100-episode moving average) for the four replay × target corners. The two buffer-equipped variants (*both*, *buffer only*) climb out of the failure floor; *target only* and *neither* flatline. Code: `DQN_Taxi.ipynb`, Part 3.

3.7 Tabular vs. DQN

Both approaches are evaluated with a pure greedy policy, so the scores are directly comparable. The small remaining gap (8.04 vs. 7.93) is within run-to-run noise; both fully solve Taxi in ∼13 steps with a 100% success rate. The real takeaway is that, for a 500-state problem, a Q-table is the simpler and more natural tool; DQN is introduced here as a *learning exercise* for how one would scale to state spaces too large to tabulate, where a function approximator becomes mandatory.

4 Conclusion

Tabular Q-learning solves Taxi reliably, and on this small, deterministic problem the choice of hyperparameters affects convergence speed more than final quality — every tuning method we tried reached a 100% success rate. The most influential knobs were `min_epsilon` and `decay_rate`, which control how decisively the agent shifts from exploration to exploitation. The DQN reaches the same behaviour through a neural network plus a replay buffer and a target network; on Taxi that is more

	Tabular Q-learning	**DQN**
Q representation	500×6 table	neural net (one-hot $\rightarrow$ MLP)
State input	integer ID (table index)	one-hot vector of length 500
Stabilisers	none needed	replay buffer + target network
Training episodes	25,000	500
Eval reward	~ 7.93 (greedy)	8.04 (greedy)
Eval steps	~ 13.1	13.0
Success rate	100%	100%

Table 7: Tabular Q-learning versus DQN on Taxi.

machinery than the problem demands, but it is the exact recipe one needs when the state space grows beyond what a table can hold. A multi-seed hyperparameter study and a replay-buffer $\times$ target-network ablation sharpen the picture. The standout configuration is a *larger* network *with weight decay* — the only one to fully solve Taxi (100%, $\sim$13 steps, near-zero variance); weight decay rescued the wide net while it hurt the small one, and dropout broke both. The RL hyperparameters — including batch size, learning frequency, and a hard-vs-soft target update — proved close to neutral, no sampled config beating the defaults. The ablation, however, is decisive: the *replay buffer* is the load-bearing component (without it the agent never learns the sparse reward), while the target network is only a modest refinement.

5 Optional extension: DQN on Pong from pixels

As an optional extension we apply the *same* DQN to Atari **Pong**, learned directly from raw screen pixels.[2] The learning algorithm barely changes — replay buffer, target network, ϵ-greedy, and a bootstrapped Bellman target all carry over, the lone refinement being a *Double*-DQN target (detailed below) — while the *representation*, the *scale*, and the *engineering* change substantially. Table 8 summarises the jump from Taxi.

5.1 Observation, state, and the four-frame Markov window

In Taxi the environment hands us a state *index*: a sufficient statistic that already satisfies the **Markov property** — the next state and reward depend only on it and the chosen action. In Pong the observation is an **image**, and a single frame is *not* Markovian: from one still you can read the ball's position but not its *direction* or *speed*, so the optimal action is genuinely undefined. We restore the Markov property the standard way — by **stacking the last four frames** as the state,

$$s_t \;=\; \big(x_{t-3},\, x_{t-2},\, x_{t-1},\, x_t\big) \in \{0,\dots,255\}^{4\times 84\times 84},$$

so that velocity (and even acceleration) becomes recoverable from differences across the stack. That four-frame window is precisely the piece Taxi got for free from its index; in Pong we have to construct it. The pipeline that produces each frame x_t: convert to greyscale, resize to 84×84, skip (and max-pool) 4 emulator frames per action, then stack 4 such frames. A **convolutional** Q-network (three conv layers, then a 512-unit dense layer, then one Q-value per action) reads the stack; rewards are clipped to $\{-1, 0, +1\}$, the 100k `uint8` buffer holds memory at $\sim$6 GB, and checkpointing lets a disconnected GPU session resume.

[2] Code: DQN_Pong.ipynb.

	Taxi DQN	**Pong DQN**
Observation	state index (one-hot of 500)	4 stacked 84×84 greyscale frames
One obs. Markov?	yes — the index is sufficient	no — stack 4 frames (see below)
Q-network	MLP (128–128)	CNN (3 conv $\rightarrow$ 512 dense), "Nature DQN"
Input size	500 discrete states	$\sim$28k pixels per state ($4\times84\times84$)
Action space	6 (all used)	$6 \rightarrow 3$ (NOOP / up / down)
Reward	native (-10 to $+20$)	clipped to $\{-1, 0, +1\}$
DQN target	vanilla (max)	**Double** DQN
Replay buffer	50k transitions	100k `uint8` images ($\sim$6 GB)
ϵ schedule	decayed over episodes	annealed over 200k frames
Target sync	every 250 steps	every 2k frames
Optimiser	Adam, `lr`= 10^{-3}	Adam, `lr`= 10^{-4}
Scale / compute	$\sim$hundreds of episodes, CPU	$\sim$1–2M frames, $\sim$1.5–2 h on GPU/MPS

Table 8: From Taxi to Pong: the learning *algorithm* is identical, but the representation (pixels vs. an index), the network (CNN vs. MLP), and the scale all change. The ϵ, action-space, and target-sync rows turned out to decide whether Pong learned at all (Section 5.3).

5.2 Vanilla vs. Double DQN

The two notebooks also differ in *one line* of the target computation. Both keep an online network θ and a target network θ^-; the question is who *selects* and who *evaluates* the next action. **Vanilla** DQN (Taxi) lets the target network do both, fused into a single max:

$$y \;=\; r + \gamma \max_{a'} Q_{\theta^-}(s', a').$$

Double DQN (Pong) splits the two roles — the online network *selects* the action, the target network *evaluates* it:

$$a^\star = \arg\max_{a'} Q_\theta(s', a'), \qquad y \;=\; r + \gamma\, Q_{\theta^-}(s', a^\star).$$

Why. The max over noisy Q-estimates is biased *upward*: it preferentially selects whichever action got lucky positive noise, so the max of the estimates sits above the max of the true values, and because DQN bootstraps, that overestimation feeds back on itself. Splitting selection from evaluation *decorrelates* the noise — if the online net over-rates an action by chance, the frozen target net usually does not over-rate that same action, so the inflation largely cancels (van Hasselt, Guez & Silver, 2016). The cost is essentially nil: both variants already maintain the two networks, so going Double is a one-line change to the target. Taxi uses vanilla because it is near-tabular and overestimation is harmless there; Pong's noisier pixel function-approximation makes the bias bite harder, so Double is the natural choice.

5.3 Results and final configuration

Our first full run never learned: at two million frames the score sat at the random-play baseline (≈ -21) from start to finish, and greedy play lost every point. Instrumenting the agent showed this was not a bug but a *configuration* that could not learn — with ϵ annealed over the first 1M of 2M frames and the full 6-action set, the policy collapsed onto a single action (near-identical Q-values everywhere) and never recovered. Two changes fix it: anneal ϵ *fast* (over 200k frames) so the agent

starts exploiting early, and expose only the three actions Pong needs (NOOP / up / down); a more frequent target sync (2k frames) helps too.

With the corrected configuration (Table 9) the *same* network and update code learn Pong from raw pixels. The average score climbs off −21 within ∼200–300k frames, reaches break-even near 1M, and a winning score by ∼2M[3]. In the saved run, the final training average over the last 20 games is 7.45; greedy evaluation over 5 held-out episodes scores 14.6 on average, and the recorded game scores 16.0. As an earlier checkpoint, the 3-action agent already averages ≈ -10 at just 600k frames — a competitive paddle that returns most balls — against the original recipe's flat −21 across the entire run.

The learning curve for this run is shown in Figure 6. The per-episode score (faint) is very noisy — individual Pong games swing widely — but the 20-episode moving average tells the story cleanly: it sits on the −21 floor for the first ∼200 episodes while the buffer fills and ϵ is still high, then climbs steadily, crosses break-even around episode ∼490, and finishes averaging $\approx +7$. In other words the agent goes from losing every point to consistently beating the built-in opponent — learned end-to-end from raw pixels.

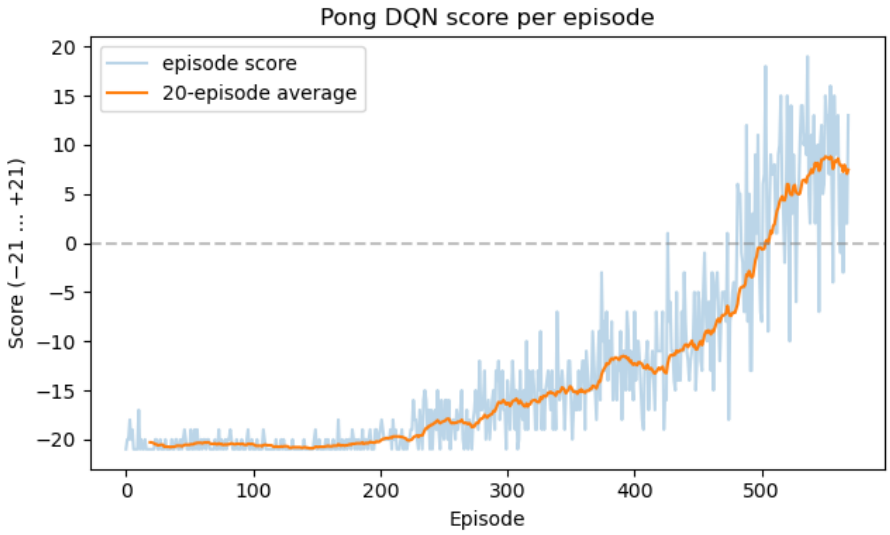

Figure 6: Pong-from-pixels learning curve for the corrected configuration (Table 9): per-episode score (faint blue) and its 20-episode moving average (orange). The average rises off the −21 random-play floor, crosses break-even (0, dashed), and ends positive — the agent learns to win from raw pixels. Code: `DQN_Pong.ipynb`, Step 10.

In short, the same DQN framework that solved Taxi also solves Pong from pixels: the core machinery — experience replay, a target network, ϵ-greedy exploration, and the bootstrapped Bellman target — carries over. The changes are the representation (a CNN over a four-frame stack), one *algorithmic* refinement — the *Double*-DQN form of the target (online net selects, target net evaluates) in place of Taxi's vanilla max — and the handful of training settings that decide whether exploration is ever converted into a winning policy.

[3]Recording of the trained agent in play: `pong.mp4`.

Setting	**Value**
Observation	$4 \times 84 \times 84$ greyscale stack (frame-skip 4)
Action set	3 — NOOP / up / down
Reward	clipped to $\{-1, 0, +1\}$
Q-network	"Nature" CNN: 3 conv $\rightarrow$ 512-unit dense
Target	Double DQN, synced every 2k frames
Optimiser	Adam, `lr`$= 10^{-4}$
Discount γ	0.99
Loss / grad clip	Huber, gradient-norm ≤ 10
Batch size	32
Replay buffer	100k `uint8` transitions ($\sim$6 GB)
Learning starts	10k frames
Train frequency	1 gradient step per 4 frames
ϵ schedule	$1.0 \rightarrow 0.02$, linear over 200k frames
Total frames	$2{\times}10^6$

Table 9: Final Pong-from-pixels configuration. The settings that turned the original flat-lining run into a learning one are the 3-action set, the fast ϵ-decay, and the 2k-frame target sync; every other line is the same DQN recipe used for Taxi, scaled up.